

```

class Triangle : public Shape
{
public:
    void SetVertex(int i, const Vector3& v);
    Vector3 GetVertex(int i) const { return m_vtx[i]; }

    virtual void Draw()
    {
        // code to draw a triangle
    }

    virtual void SetId(int id)
    {
        // call base class' implementation
        Shape::SetId(id);

        // do additional work specific to Triangles...
    }

private:
    Vector3 m_vtx[3];
};

// -----

void main(int, char**)
{
    Shape* pShape1 = new Circle;
    Shape* pShape2 = new Triangle;

    pShape1->Draw();
    pShape2->Draw();

    // ...
}

```

3.4 Computer Hardware Fundamentals

Programming in a high-level language like C++, C# or Python is an efficient way to build software. But the higher-level your language is, the more it shields you from the underlying details of the hardware on which your code runs. To become a truly proficient programmer, it's important to understand the architecture of your target hardware. This knowledge can help you to op-

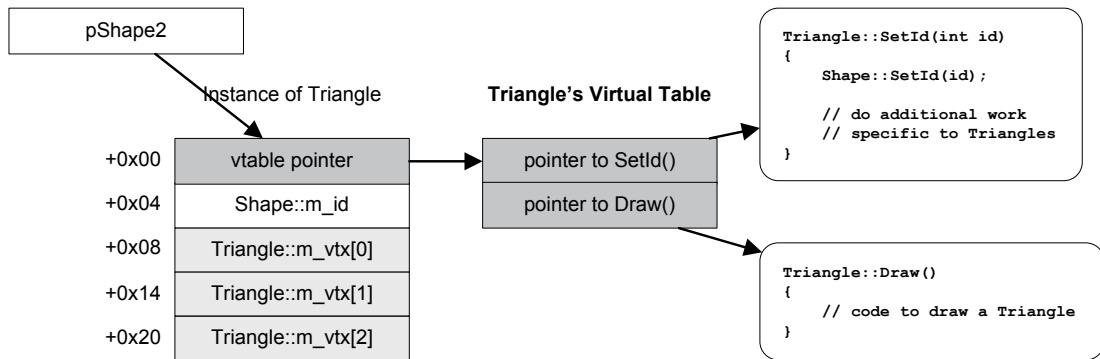


Figure 3.18. `pShape2` points to an instance of class `Triangle`.

timize your code. It's also crucial for concurrent programming—and all programmers must understand concurrency if they hope to take full advantage of the ever-increasing degree of parallelism in modern computing hardware.

3.4.1 Learning from the Simpler Computers of Yesteryear

In the following pages, we'll discuss the design of a simple, generic CPU, rather than diving into the specifics of any one particular processor. However, some readers may find it helpful to ground our somewhat theoretical discussion by reading about the specifics of a real CPU. I myself learned how computers work when I was a teenager, by programming my Apple II computer and the Commodore 64. Both of these machines had a simple CPU known as the 6502, which was designed and manufactured by MOS Technology Inc. I also learned some of what I know by reading about and working with the common ancestor to Intel's entire x86 line of CPUs, the 8086 (and its cousin the 8088). Both of these processors are great for learning purposes due to their simplicity. This is especially true of the 6502, which is the simplest CPU I've ever worked with. Once you understand how a 6502 and/or an 8086 works, modern CPUs will be a lot easier to understand.

To that end, here are a few great resources on the details of 6502 and 8086 architecture and programming:

- Chapters 1 and 2 of the book "Inside the Apple IIe" by Gary B. Little [33] provides a great overview of 6502 assembly language programming. The book is available online at <http://www.apple2scans.net/files/InsidetheIIe.pdf>.

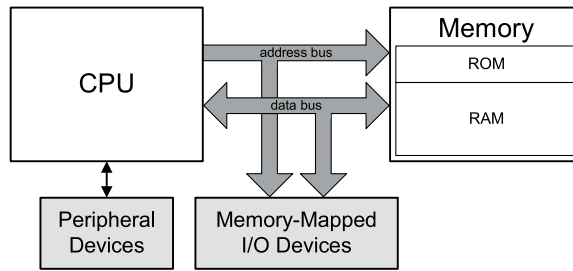


Figure 3.19. A simple serial von Neumann computer architecture.

- <http://flint.cs.yale.edu/cs421/papers/x86-asm/asm.html> gives a nice overview of the x86 instruction set architecture.
- You should also definitely check out *Michael Abrash's Graphics Programming Black Book* [1] for loads of useful information about assembly programming on the 8086, plus incredible tips for software optimization and graphics programming back in the early days of gaming.

3.4.2 Anatomy of a Computer

The simplest computer consists of a *central processing unit* (CPU) and a bank of *memory*, connected to one another on a circuit board called the *motherboard* via one or more *buses*, and connected to external peripheral devices by means of a set of *I/O ports* and/or *expansion slots*. This basic design is referred to as the *von Neumann architecture* because it was first described in 1945 by the mathematician and physicist John von Neumann and his colleagues while working on the classified ENIAC project. A simple serial computer architecture is illustrated in Figure 3.19.

3.4.3 CPU

The CPU is the “brains” of the computer. It consists of the following components:

- an *arithmetic/logic unit* (ALU) for performing integer arithmetic and bit shifting,
- a *floating-point unit* (FPU) for doing floating-point arithmetic (typically using the IEEE 754 floating-point standard representation),
- virtually all modern CPUs also contain a *vector processing unit* (VPU) which is capable of performing floating-point and integer operations on multiple data items in parallel,

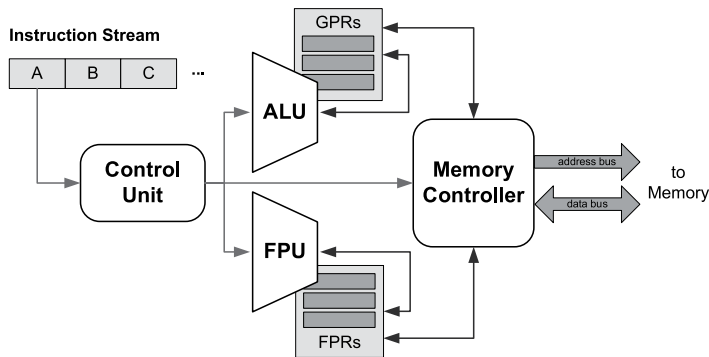


Figure 3.20. Typical components of a serial CPU.

- a *memory controller* (MC) or *memory management unit* (MMU) for interfacing with on-chip and off-chip memory devices,
- a bank of *registers* which act as temporary storage during calculations (among other things), and
- a *control unit* (CU) for decoding and dispatching machine language instructions to the other components on the chip, and routing data between them.

All of these components are driven by a periodic square wave signal known as the *clock*. The frequency of the clock determines the rate at which the CPU performs operations such as executing instructions or performing arithmetic. The typical components of a serial CPU are shown in Figure 3.20.

3.4.3.1 ALU

The ALU performs unary and binary arithmetic operations such as negation, addition, subtraction, multiplication and division, and also performs logical operations such as AND, OR, exclusive OR (abbreviated XOR or EOR), bitwise complement and bit shifting. In some CPU designs, the ALU is physically split into an arithmetic unit (AU) and a logic unit (LU).

An ALU typically performs integer operations only. Floating-point calculations require very different circuitry, and are usually performed by a physically separate floating-point unit (FPU). Early CPUs like the Intel 8088/8086 had no on-chip FPU; if floating-point math support was desired, these CPUs had to be augmented with a separate FPU co-processor chip such as the Intel 8087. In later CPU designs, an FPU was typically included on the main CPU die.

3.4.3.2 VPU

A vector processing unit (VPU) acts a bit like a combination ALU/FPU, in that it can typically perform both integer and floating-point arithmetic. What differentiates a VPU is its ability to apply arithmetic operators to *vectors* of input data—each of which typically consists of between two and 16 floating-point values or up to 64 integer values of various widths—rather than to scalar inputs. Vector processing is also known as *single instruction multiple data* or SIMD, because a single arithmetic operator (e.g., multiplication) is applied to multiple pairs of inputs simultaneously. See Section 4.10 for more details.

Today’s CPUs don’t actually contain an FPU *per se*. Instead, all floating-point calculations, even those involving scalar `float` values, are performed by the VPU. Eliminating the FPU reduces the transistor count on the CPU die, allowing those transistors to be used to implement larger caches, more-complex out-of-order execution logic, and so on. And as we’ll see in Section 4.10.6, optimizing compilers will typically convert math performed on `float` variables into *vectorized* code that uses the VPU anyway.

3.4.3.3 Registers

In order to maximize performance, an ALU or FPU can usually only perform calculations on data that exists in special high-speed memory cells called *registers*. Registers are typically physically separate from the computer’s main memory, located on-chip and in close proximity to the components that access them. They’re usually implemented using fast, high-cost multi-ported static RAM or SRAM. (See Section 3.4.5 for more information about memory technologies.) A bank of registers within a CPU is called a *register file*.

Because registers aren’t part of main memory,¹ they typically don’t have addresses but they do have names. These could be as simple as R0, R1, R2 etc., although early CPUs tended to use letters or short mnemonics as register names. For example, the Intel 8088/8086 had four 16-bit general-purpose registers named AX, BX, CX and DX. The 6502 by MOS Technology, Inc. performed all of its arithmetic operations using a register known as the *accumulator*² (A), and used two auxiliary registers called X and Y for other operations such as indexing into an array.

¹Some early computers did use main RAM to implement registers. For example, the 32 registers in the IBM 7030 Stretch (IBM’s first transistor-based supercomputer) were “overlaid” on the first 32 addresses in main RAM. In some early ALU designs, one of its inputs would come from a register while the other came from main RAM. These designs were practical because back then, RAM access latencies were low relative to the overall performance of the CPU.

²The term “accumulator” arose because early ALUs used to work one bit at a time, and would therefore *accumulate* the answer by masking and shifting the individual bits into the result register.

Some of the registers in a CPU are designed to be used for general calculations. They're appropriately named *general-purpose registers* (GPR). Every CPU also contains a number of *special-purpose registers* (SPR). These include:

- the *instruction pointer* (IP),
- the *stack pointer* (SP),
- the *base pointer* (BP) and
- the *status register*.

Instruction Pointer

The *instruction pointer* (IP) contains the address of the currently-executing instruction in a machine language program (more on machine language in Section 3.4.7.2).

Stack Pointer

In Section 3.3.5.2 we saw how a program's *call stack* serves both as the primary mechanism by which functions call one another, and as the means by which memory is allocated for local variables. The *stack pointer* (SP) contains the address of the top of the program's call stack. The stack can grow up or down in terms of memory addresses, but for the purpose of this discussion let's assume it grows downward. In this case, a data item may be *pushed* onto the stack by subtracting the size of the item from the value of the stack pointer, and then writing the item at the new address pointed to by SP. Likewise, an item can be *popped* off the stack by reading it from the address pointed to by SP, and then adding its size to SP.

Base Pointer

The *base pointer* (BP) contains the base address of the current function's *stack frame* on the call stack. Many of a function's local variables are allocated within its stack frame, although the optimizer may assign others exclusively to a register for the duration of the function. Stack-allocated variables occupy a range of memory addresses at a unique offset from the base pointer. Such a variable can be located in memory by simply subtracting its unique offset from the address stored in BP (assuming the stack grows down).

Status Register

A special register known as the *status register*, *condition code register* or *flags*

register contains bits that reflect the results of the most-recent ALU operation. For instance, if the result of a subtraction is zero, the *zero bit* (typically named “Z”) is set within the status register, otherwise the bit is cleared. Likewise, if an add operation resulted in an overflow, meaning that a binary 1 must be “carried” to the next word of a multi-word addition, the *carry bit* (often named “C”) is set, otherwise it is cleared.

The flags in the status register can be used to control program flow via *conditional branching*, or they can be used to perform subsequent calculations, such as adding the carry bit to the next word in a multi-word addition.

Register Formats

It’s important to understand that the FPU and VPU typically operate on their own private sets of registers, rather than making use of the ALU’s general-purpose integer registers. One reason for this is speed—the closer the registers are to the compute unit that uses them, the less time is needed to access the data they contain. Another reason is that the FPU’s and VPU’s registers are typically *wider* than the ALU’s GPRs.

For example, a 32-bit CPU has GPRs that are 32 bits wide each, but an FPU might operate on 64-bit double-precision floats, or even 80-bit “extended” double-precision values, meaning that its registers have to be 64 or 80 bits wide, respectively. Likewise, each register used by a VPU needs to contain a *vector* of input data, meaning that these registers must be much wider than a typical GPR. For example, Intel’s SSE2 (streaming SIMD extensions) vector processor can be configured to perform calculations on vectors containing either four single-precision (32-bit) floating-point values each, or two double-precision (64-bit) values each. Hence SSE2 vector registers are each 128 bits wide.

The physical separation of registers between ALU and FPU is one reason why conversions between `int` and `float` were very expensive, back in the days when FPUs were commonplace. Not only did the bit pattern of each value have to be converted back and forth between its two’s complement integer format and its IEEE 754 floating-point representation, but the data also had to be transferred physically between the general-purpose integer registers and the FPU’s registers. However, today’s CPUs no longer contain an FPU per se—all floating-point math is typically performed by a vector processing unit. A VPU can handle both integer and floating-point math, and conversions between the two are much less expensive, even when moving data from an integer GPU into a vector register or vice-versa. That said, it’s still a good idea to avoid converting data between `int` and `float` formats

where possible, because even a low-cost conversion is still more expensive than no conversion.

3.4.3.4 Control Unit

If the CPU is the “brains” of the computer, then the *control unit* (CU) is the “brains” of the CPU. Its job is to manage the flow of data within the CPU, and to orchestrate the operation of all of the CPU’s other components.

The CU runs a program by reading a stream of *machine language* instructions, decoding each instruction by breaking it into its opcode and operands, and then issuing work requests and/or routing data to the ALU, FPU, VPU, the registers and/or the memory controller as dictated by the current instruction’s opcode. In pipelined and superscalar CPUs, the CU also contains complex circuitry to handle branch prediction and the scheduling of instructions for out-of-order execution. We’ll look at the operation of the CU in more detail in Section 3.4.7.

3.4.4 Clock

Every digital electronic circuit is essentially a state machine. In order for it to change states, the circuit must be driven to do so by a digital signal. Such a signal might be provided by *changing the voltage* on a line in the circuit from 0 volts to 3.3 volts, or vice-versa.

State changes within a CPU are typically driven by a periodic square wave signal known as the *system clock*. Each rising or falling edge of this signal is known as a *clock cycle*, and the CPU can perform at least one primitive operation on every cycle. To a CPU, time therefore appears to be *quantized*.³

The rate at which a CPU can perform its operations is governed by the *frequency* of the system clock. Clock speeds have increased significantly over the past few decades. Early CPUs developed in the 1970s, like the MOS Technology’s 6502 and Intel’s 8086/8088 CPUs, had clocks that ran in the 1–2 MHz range (millions of cycles per second). Today’s CPUs, like the Intel Core i7, are typically clocked in the 2–4 GHz range (*billions* of cycles per second).

It’s important to realize that one CPU instruction doesn’t necessarily take one clock cycle to execute. Not all instructions are created equal—some instructions are very simple, while others are more complex. Some instructions are implemented under the hood as a combination of simpler micro-operations (μ -ops) and therefore take many more cycles to execute than their simpler counterparts.

³Contrast this to an *analog* electronic circuit, where time is treated as *continuous*. For example, an old-school signal generator can produce a true sine wave that smoothly varies between, say, –5 volts and 5 volts over time.

Also, while early CPUs could truly execute some instructions in a single clock cycle, today's pipelined CPUs break even the simplest instruction down into multiple *stages*. Each stage in a pipelined CPU takes one clock cycle to execute, meaning that a CPU with an N -stage pipeline has a minimum *instruction latency* of N clock cycles. A simple pipelined CPU can retire instructions at a *rate* of one instruction per clock cycle, because a new instruction is fed into the pipeline each clock tick. But if you were to trace one particular instruction through the pipeline, it would take N cycles to move from start to finish through the pipeline. We'll discuss pipelined CPUs in more depth in Section 4.2.

3.4.4.1 Clock Speed versus Processing Power

The “processing power” of a CPU or computer can be defined in various ways. One common measure is the *throughput* of the machine—the number of operations it can perform during a given interval of time. Throughput is expressed either in units of millions of instructions per second (MIPS) or floating-point operations per second (FLOPS).

Because instructions or floating-point operations don't generally complete in exactly one cycle, and because different instructions take differing numbers of cycles to run, the MIPS or FLOPS metrics of a CPU are just averages. As such, you cannot simply look at the clock frequency of a CPU and determine its processing power in MIPS or FLOPS. For example, a serial CPU running at 3 GHz, in which one floating-point multiply takes six cycles to complete on average, could theoretically achieve 0.5 GFLOPS. But many factors including pipelining, superscalar designs, vector processing, multicore CPUs and other forms of parallelism conspire to obfuscate the relationship between clock speed and processing power. Thus the only way to determine the true processing power of a CPU or computer is to measure it—typically by running standardized benchmarks.

3.4.5 Memory

The *memory* in a computer acts like a bank of mailboxes at the post office, with each box or “cell” typically containing a single *byte* of data (an eight-bit value).⁴ Each one-byte memory cell is identified by its *address*—a simple numbering

⁴Actually, early computers often accessed memory in units of “words” that were larger than 8 bits. For example, the IBM 701 (produced in 1952) addressed memory in units of 36-bit words, and the PDP-1 (made in 1959) could access up to 4096 18-bit memory words. Eight-bit bytes were popularized by the Intel 8008 in 1972. Seven bits are required to encode both lower- and uppercase English letters. By extending this to eight bits, a wide range of special characters could also be supported.

scheme ranging from 0 to $N - 1$, where N is the size of addressable memory in bytes.

Memory comes in two basic flavors:

- *read-only memory* (ROM), and
- read/write memory, known for historical reasons⁵ as *random access memory* (RAM).

ROM modules retain their data even when power is not applied to them. Some types of ROM can be programmed only once. Others, known as *electronically erasable programmable ROM* or EEPROM, can be reprogrammed over and over again. (Flash drives are one example of EEPROM memory.)

RAM can be further divided into *static* RAM (SRAM) and *dynamic* RAM (DRAM). Both static and dynamic RAM retain their data as long as power is applied to them. But unlike static RAM, dynamic RAM also needs to be “refreshed” periodically (by reading the data and then re-writing it) in order to prevent its contents from disappearing. This is because DRAM memory cells are built from MOS capacitors that gradually lose their charge, and reading such memory cells is destructive to the data they contain.

RAM can also be categorized by various other design characteristics, such as:

- whether it is *multi-ported*, meaning that it be accessed simultaneously by more than one component within the CPU;
- whether it operates by being synchronized to a clock (SDRAM) or asynchronously;
- and whether or not it supports double data rate access (DDR), meaning the RAM can be read or written on both rising and falling clock edges.

3.4.6 Buses

Data is transferred between the CPU and memory over connections known as *buses*. A bus is just a bundle of parallel digital “wires” called *lines*, each of which can represent a single bit of data. When the line carries a voltage signal⁶ it represents a binary one, and when the line has no voltage applied

⁵Random-access memory was so named because earlier memory technologies used delay loops to store data, meaning that it could only be read in the order it was written. RAM technology improved on this situation by permitting data to be accessed at random, i.e., in any order.

⁶Early *transistor-transistor* logic (TTL) devices operated at a supply voltage of 5 volts, so a 5-volt signal would represent a binary 1. Most of today’s digital electronic devices utilize *complementary metal oxide semiconductor* logic (CMOS) which can operate at a lower supply voltage, typically between 1.2 and 3.3 volts.

(0 volts) it represents a binary zero. A bundle of n single-bit lines arranged in parallel can transmit an n -bit number (i.e., any number in the range 0 through $2^n - 1$).

A typical computer contains two buses: An *address bus* and a *data bus*. The CPU loads data from a memory cell into one of its registers by supplying an address to the memory controller via the address bus. The memory controller responds by presenting the bits of the data item stored in the cell(s) in question onto the data bus, where it can be “seen” by the CPU. Likewise, the CPU writes a data item to memory by broadcasting the destination address over the address bus and placing the bit pattern of the data item to write onto the data bus. The memory controller responds by writing the given data into the corresponding memory cell(s). We should note that the address and data buses are sometimes implemented as two physically separate sets of wires, and sometimes as a single set of wires which are multiplexed between address and data bus functions during different phases of the memory access cycle.

3.4.6.1 Bus Widths

The *width* of the *address bus*, measured in bits, controls the range of possible addresses that can be accessed by the CPU (i.e., the size of *addressable memory* in the machine). For example, a computer with a 16-bit address bus can access at most 64 KiB of memory, using addresses in the range 0x0000 through 0xFFFF. A computer with a 32-bit address bus can access a 4 GiB memory, using addresses in the range 0x00000000 through 0xFFFFFFFF. And a machine with a 64-bit address bus can access a staggering 16 EiB (exbibytes) of memory. That’s $2^{64} = 16 \times 1024^6 \approx 1.8 \times 10^{19}$ bytes!

The width of the *data bus* determines how much data can be transferred between CPU registers and memory at a time. (The data bus is typically the same width as the general-purpose registers in the CPU, although this isn’t always the case.) An 8-bit data bus means that data can be transferred one byte at a time—loading a 16-bit value from memory would require two separate memory cycles, one to fetch the least-significant byte and one to fetch the most-significant byte. At the other end of the spectrum, a 64-bit data bus can transfer data between memory and a 64-bit register as a single memory operation.

It’s possible to access data items that are narrower than the width of a machine’s data bus, but it’s typically more costly than accessing items whose widths match that of the data bus. For example, when reading a 16-bit value on a 64-bit machine, a full 64 bits worth of data must still be read from memory. The desired 16-bit field then has to be masked off and possibly shifted into place within the destination register. This is one reason why the C language

does not define an `int` to be a specific number of bits wide—it was purposefully defined to match the “natural” size of a word on the target machine, in an attempt to make source code more portable. (Ironically this policy has actually resulted in source code often being *less* portable due to implicit assumptions about the width of an `int`.)

3.4.6.2 Words

The term “word” is often used to describe a multi-byte value. However, the number of bytes comprising a word is not universally defined. It depends to some degree on context.

Sometimes the term “word” refers to the smallest multi-byte value, namely 16 bits or two bytes. In that context, a double word would be 32 bits (four bytes) and a quad word would be 64 bits (eight bytes). This is the way the term “word” is used in the Windows API.

On the other hand, the term “word” is also used to refer to the “natural” size of data items on a particular machine. For example, a machine with 32-bit registers and a 32-bit data bus operates most naturally with 32-bit (four byte) values, and programmers and hardware folks will sometimes say that such a machine as a *word size* of 32 bits. The takeaway here is to be aware of context whenever you hear the term “word” being used to refer to the size of a data item.

3.4.6.3 *n*-Bit Computers

You may have encountered the term “*n*-bit computer.” This usually means a machine with an *n*-bit data bus and/or registers. But the term is a bit ambiguous, because it might also refer to a computer whose address bus is *n* bits wide. Also, on some CPUs the data bus and register widths don’t match. For example, the 8088 had 16-bit registers and a 16-bit address bus, but it only had an 8-bit data bus. Hence it acted like a 16-bit machine internally, but its 8-bit data bus caused it to behave like an 8-bit machine in terms of memory accesses. Again, be aware of context when talking about an *n*-bit machine.

3.4.7 Machine and Assembly Language

As far as the CPU is concerned, a “program” is nothing more than a sequential stream of relatively simple *instructions*. Each instruction tells the control unit (CU), and ultimately the other components within the CPU such as the memory controller, ALU, FPU or VPU, to perform an operation. An instruction might move data around within the computer or within the CPU itself, or it might transform that data in some manner (e.g., by performing an arith-

metic or logical operation on the data). Normally the instructions in a program are executed sequentially, although some instructions can alter this sequential flow of control by “jumping” to a new spot within the program’s overall instruction stream.

3.4.7.1 Instruction Set Architecture (ISA)

CPU designs vary widely from one manufacturer to the next. The set of all instructions supported by a given CPU, along with various other details of the CPU’s design like its addressing modes and the in-memory instruction format, is called its *instruction set architecture* or ISA. (This is not to be confused with a programming language’s *application binary interface* or ABI, which defines higher-level protocols like calling conventions.) We won’t attempt to cover the details of any one CPU’s ISA here, but the following categories of instruction types are common to pretty much every ISA:

- *Move*. These instructions move data between registers, or between memory and a register. Some ISAs break the “move” instruction into separate “load” and “store” instructions.
- *Arithmetic operations*. These of course include addition, subtraction, multiplication and division, but may also include other operations like unary negation, inversion, square root, and so on.
- *Bitwise operators*. These include AND, OR, exclusive OR (abbreviated XOR or EOR) and bitwise complement.
- *Shift/rotate operators*. These instructions allow the bits within a data word to be shifted left or right, with or without affecting the carry bit in the status register, or rotated (where the bits rolling off one end of the word “wrap around” to the other end).
- *Comparison*. These instructions allow two values to be compared, in order to determine if one is less than, greater than or equal to the other. In most CPUs, the comparison instructions use the ALU to subtract the two input values, thereby setting the appropriate bits in the status register, but the result of the subtraction is simply discarded.
- *Jump and branch*. These allow program flow to be altered by storing a new address into the instruction pointer. This can be done either unconditionally (in which case it is called a “jump” instruction) or conditionally based on the state of various flags in the status register (in which case it is often called a “branch”). For example, a “branch if zero” instruction alters the contents of the IP if and only if the “Z” bit in the status register is set.

- *Push and pop.* Most CPUs provide special instructions for pushing the contents of a register onto the program stack and for popping the current value at the top of the stack into a register.
- *Function call and return.* Some ISAs provide explicit instructions for calling a function (also known as a procedure or subroutine) and returning from it. However, function call and return semantics can also be provided by a combination of push, pop and jump instructions.
- *Interrupts.* An “interrupt” instruction triggers a digital signal within the CPU that causes it to jump temporarily to a pre-installed *interrupt service routine* routine which is often not part of the program being run. Interrupts are used to notify the operating system or a user program of events such as an input becoming available on a peripheral device. Interrupts can also be triggered by user programs in order to “call” into the operating system kernel’s routines. See section 4.4.2 for more details.
- *Other instruction types.* Most ISAs support a variety of instruction types that don’t fall into one of the categories listed above. For example, the “no-op” instruction (often called NOP) is an instruction that has no effect other than to introduce a short delay. NOP instructions also consume memory, and on some ISAs they are used to align subsequent instructions properly in memory.

We can’t possibly list all instruction types here, but if you’re curious you can always read through the ISA documentation of a real processor like the Intel x86 (which is available at <http://intel.ly/2woVFQ8>).

3.4.7.2 Machine Language

Computers can only deal with numbers. As such, each instruction in a program’s instruction stream must be encoded numerically. When a program is encoded in this way, we say it is written in *machine language*, or ML for short. Of course, machine language isn’t a single language—it’s really a multitude of languages, one for each distinct CPU/ISA.

Every machine language instruction is comprised of three basic parts:

- an *opcode*, which tells the CPU which operation to perform (add, subtract, move, jump, etc.),
- zero or more *operands* which specify the inputs and/or outputs of the instruction, and
- some kind of *options field*, specifying things like the *addressing mode* of the instruction and possibly other flags.

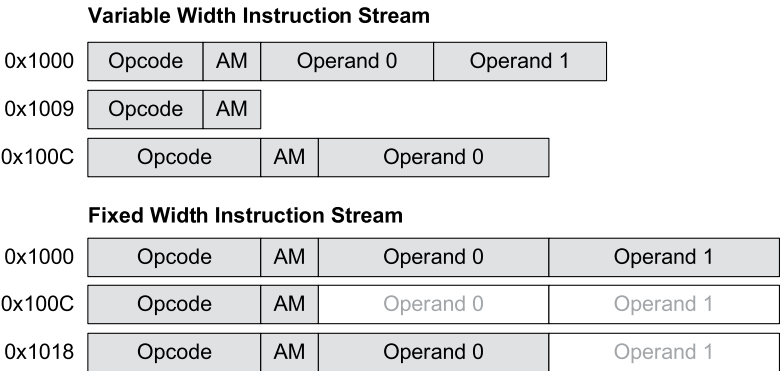


Figure 3.21. Two hypothetical machine language instruction encoding schemes. Top: In a variable-width encoding scheme, different instructions may occupy different numbers of bytes in memory. Bottom: A fixed-width instruction encoding uses the same number of bytes for every instruction in an instruction stream.

Operands come in many flavors. Some instructions might take the names (encoded as numeric ids) of one or more registers as operands. Others might expect a literal value as an operand (e.g., “load the value 5 into register R2,” or “jump to address 0x0102ED5C”). The way in which an instruction’s operands are interpreted and used by the CPU is known as the instruction’s *addressing mode*. We’ll discuss addressing modes in more detail in Section 3.4.7.4.

The opcode and operands (if any) of an ML instruction are packed into a contiguous sequence of bits called an *instruction word*. A hypothetical CPU might encode its instructions as shown in Figure 3.21 with perhaps the first byte containing the opcode, addressing mode and various options flags, followed by some number of bytes for the operands. Each ISA defines the width of an instruction word (i.e., the number of bits occupied by each instruction) differently. In some ISAs, all instructions occupy a fixed number of bits; this is typical of *reduced instruction set computers* (RISC). In other ISAs, different types of instructions may be encoded into differently-sized instruction words; this is common in *complex instruction set computers* (CISC).

Instruction words can be as small as four bits in some microcontrollers, or they may be many bytes in size. Instruction words are often multiples of 32 or 64 bits, because this matches the width of the CPU’s registers and/or data bus. In *very long instruction word* (VLIW) CPU designs, parallelism is achieved by allowing multiple operations to be encoded into a single very wide instruction word, for the purpose of executing them in parallel. Instructions in a VLIW ISA can therefore be upwards of hundreds of bytes in width.


```
ReturnZero:
    ; else return 0;
    xor    eax, eax    ; set EAX to zero
    ret    ; (EAX is the return value)
```

Let's break this down. The `cmp` instruction compares the values in registers EAX and EBX (which we're assuming contain the values `a` and `b` from the C snippet). Next, the `jle` instruction branches to the label `ReturnZero`, but it does so if and *only if* the value in EAX is less than or equal to EBX. Otherwise it falls through.

If EAX is *greater* than EBX ($a > b$), we fall through to the `add` instruction, which calculates $a + b$ and stores the result back into EAX, which we're assuming serves as the return value. We issue a `ret` instruction, and control is returned to the calling function.

If EAX is less than or equal to EBX ($a \leq b$), the branch is taken and we continue execution immediately after the label `ReturnZero`. Here, we use a little trick to set EAX to zero by XORing it with itself. Then we issue a `ret` instruction to return that zero to the caller.

For more details on Intel assembly language programming, see <http://www.cs.virginia.edu/~evans/cs216/guides/x86.html>.

3.4.7.4 Addressing Modes

A seemingly simple instruction like “move” (which transfers data between registers and memory) has many different variants. Are we moving a value from one register to another? Are we loading a literal value like 5 into a register? Are we loading a value from memory into a register? Or are we storing a value in a register out to memory? All of these variants are called *addressing modes*. We won't go over all the possible addressing modes here, but the following list should give you a good feel for the kinds of addressing modes you will encounter on a real CPU:

- *Register addressing*. This mode allows values to be transferred from one register to another. In this case, the operands of the instruction specify which registers are involved in the operation.
- *Immediate addressing*. This mode allows a literal or “immediate” value to be loaded into a register. In this case, the operands are the target register and the immediate value to be loaded.
- *Direct addressing*. This mode allows data to be moved to or from memory. In this case, the operands specify the direction of the move (to or from memory) and the memory address in question.